PROJET : MOUL HANOUT

CAHIER DES CHARGES

Transformation Digitale des Epiceries Traditionnelles Marocaines

Version	Auteur(s)	Date d'édition	Validateur	Date validation
1.0	BAGHDAD Mohamed BAAKKA Monssef	01/04/2026	Encadrant PFA	16/04/2026

Réalisé par : BAAKKA Monssef et BAGHDAD Mohamed
Année universitaire : 2025-2026

TABLE DES MATIÈRES

TABLE DES MATIÈRES 2
1. Introduction 4
2. Contexte et Problématique 6
2.1 Contexte 6
2.2 Problématique 6
3. Objectifs du Projet 7
3.1 Objectif Général 7
3.2 Objectifs Spécifiques 7
4. Périmètre du Projet 8
4.1 Périmètre Fonctionnel 8
4.2 Hors Périmètre 8
4.3 Contraintes du Projet 8
5. Identification des Acteurs 9
6. Description Générale du Système 10
7. Besoins Fonctionnels 11
7.1 Module d'Authentification et Gestion des Comptes 11
Fonctionnalités 11
Règles de Gestion 11
7.2 Module de Gestion du Catalogue Produits 11
Fonctionnalités 11
Règles de Gestion 11
7.3 Module Point de Vente (POS) 11
Fonctionnalités 12
Règles de Gestion 12
7.4 Module de Gestion des Stocks (Inventaire Entrées/Sorties) 12
Fonctionnalités 12
Règles de Gestion 12
7.5 Module d'Alertes 13
7.5.1 Alertes de Rupture de Stock 13
7.5.2 Alertes de Date de Péremption 13
Règles de Gestion 13
7.6 Module Tableau de Bord et Rapports 13
Fonctionnalités 13
Règles de Gestion 14
7.7 Module Administration 14
Fonctionnalités 14
Règles de Gestion 14
8. User Stories 15

8.1 User Stories – Gérant 15
8.2 User Stories – Caissier 15
8.3 User Stories – Système (Alertes Automatiques) 15
9. Besoins Non Fonctionnels 17
9.1 Sécurité 17
9.2 Performance 17
9.3 Disponibilité et Fiabilité 17
9.4 Ergonomie et Accessibilité 17
9.5 Maintenabilité et Évolutivité 17
9.6 Compatibilité 17
10. Architecture Technique 19
10.1 Vue d'Ensemble 19
10.2 Couche Présentation (Frontend) 19
10.3 Couche Applicative (Backend) 19
10.4 Couche Données 19
10.5 Infrastructure et DevOps 19
11. Modélisation de la Base de Données 20
11.1 Entités Principales 20
11.2 Relations Clés 20
12. Planification du Projet 21
12.1 Phases du Projet 21
12.2 Estimation Temporelle 21
13. Gestion des Risques 22
14. Conclusion 23
15. Conception – Diagrammes UML 24
15.1 Diagramme de Cas d'Utilisation (Use Case) 24
15.2 Diagramme de Classes 25
15.3 Diagramme de Séquence – Connexion (Login) 26
15.4 Diagramme de Séquence – Vente, Paiement et Reçu 26

TABLES DES FIGURES

Figure 1: Diagramme de cas d'utilisation 24
Figure 2: Diagramme des classes 25
Figure 3: Diagramme de Séquence – Connexion (Login) 26
Figure 4: Diagramme de Séquence – Vente, Paiement et Reçu 27

1. Introduction

Le commerce de proximité occupe une place centrale dans l'économie informelle et formelle du Maroc. Les épiceries de quartier, communément appelées « Hanout », constituent le premier point de vente pour des millions de foyers marocains, en assurant un accès rapide et quotidien aux produits de première nécessité. Ce type de commerce joue également un rôle social important, en créant un lien direct entre le commerçant et sa clientèle locale.

Cependant, malgré leur importance économique et sociale, ces commerces fonctionnent encore majoritairement selon des méthodes traditionnelles et peu structurées : gestion des stocks basée sur la mémoire ou des notes manuscrites, encaissements réalisés sans système informatisé, absence de suivi des ventes et manque total de traçabilité des mouvements de marchandises. Cette situation engendre plusieurs problématiques, notamment des erreurs de gestion, des pertes financières, des ruptures de stock fréquentes, ainsi qu'une difficulté à prendre des décisions basées sur des données fiables.

C'est dans ce contexte que s'inscrit le projet Moul Hanout, une plateforme de gestion digitale conçue spécifiquement pour répondre aux besoins des épiceries et des commerces de petite et moyenne taille au Maroc. Cette solution vise à accompagner la transformation digitale de ces commerces en leur fournissant un système simple, accessible et adapté à leur réalité terrain. Elle ambitionne de doter le commerçant d'un Point de Vente (POS) moderne, d'un système de gestion et de suivi des stocks en temps réel, ainsi que d'un système d'alertes intelligent permettant d'anticiper les ruptures de stock et de gérer efficacement les dates de péremption.

Au-delà de l'aspect opérationnel, le projet Moul Hanout s'inscrit également dans une logique d'optimisation de la performance des commerces, en facilitant l'accès à des indicateurs clés tels que le chiffre d'affaires, les produits les plus vendus ou encore les marges réalisées. Il constitue ainsi un levier stratégique pour améliorer la rentabilité, la prise de décision et la compétitivité des commerces de proximité.

Le présent document constitue le Cahier des Charges (CDC) du projet Moul Hanout. Il a pour objectif de fournir une description détaillée et structurée du système à développer, incluant les objectifs du projet, les besoins fonctionnels et non fonctionnels, l'architecture technique envisagée, la modélisation des données, ainsi que la planification du projet et l'identification des risques potentiels.

2. Contexte et Problématique

2.1 Contexte

Le Maroc compte plus de 150 000 épiceries de proximité réparties sur l'ensemble du territoire national. Ces commerces représentent un tissu économique vital, notamment dans les zones urbaines denses et les quartiers populaires. Cependant, leur mode de gestion reste archaïque :

- Gestion des stocks entièrement manuelle, souvent basée sur la mémoire du gérant
- Absence de caisse enregistreuse structurée, les ventes étant notées sur cahier ou mémorisées
- Aucun suivi des dates de péremption, engendrant des risques sanitaires et des pertes financières
- Aucune visibilité sur la performance commerciale (produits phares, marges, tendances)
- Ruptures de stock fréquentes faute de système d'alerte préventive

2.2 Problématique

Face à ces constats, la problématique centrale du projet est la suivante :

> ***« Comment concevoir une solution digitale simple, adaptée aux contraintes des petits commerçants marocains, permettant de gérer les ventes, le stock, les alertes et les performances commerciales, sans nécessiter de compétences informatiques avancées ? »***

Les enjeux identifiés sont multiples : réduction des pertes dues aux péremptions, élimination des ruptures de stock imprévisibles, automatisation des tâches répétitives de comptabilité, et amélioration de la rentabilité globale du commerce.

3. Objectifs du Projet

3.1 Objectif Général

L'objectif général du projet Moul Hanout est de concevoir et développer une plateforme web de gestion commerciale complète, accessible et intuitive, destinée aux épiceries de petite et moyenne taille au Maroc. Cette plateforme permettra aux commerçants de digitaliser leur gestion des ventes, de leur stock et de leurs alertes produits, en s'appuyant sur une architecture technique moderne et robuste.

3.2 Objectifs Spécifiques

- Offrir un Point de Vente (POS) complet : enregistrement des ventes, gestion des paiements, édition de tickets de caisse
- Mettre en place un système de suivi des entrées et sorties de stock en temps réel
- Implémenter un système d'alertes automatiques pour les produits en rupture imminente (seuil configurable)
- Implémenter un système d'alertes pour les produits dont la date de péremption approche
- Fournir un tableau de bord analytique permettant à l’admin de piloter son activité
- Gérer plusieurs rôles d'utilisateurs (Gérant, Caissier) avec des droits différenciés
- Assurer la sécurité et la confidentialité des données commerciales
- Proposer une interface en français adaptée au contexte marocain

4. Périmètre du Projet

4.1 Périmètre Fonctionnel

La plateforme Moul Hanout couvrira les fonctionnalités suivantes dans le cadre de ce projet :

- Authentification et gestion des sessions (Admin / Caissier)
- Gestion du catalogue produits (création, modification, suppression, catégorisation)
- Point de Vente (POS) : interface de caisse, sélection de produits, calcul du total, enregistrement de la transaction
- Gestion des stocks : suivi des niveaux, enregistrement des entrées (approvisionnement) et sorties (ventes)
- Alertes de stock bas : notification automatique lorsqu'un produit descend sous le seuil minimal configurable
- Alertes de date de péremption : notification configurable (J-7, J-3, J-1 avant expiration)
- Tableau de bord : statistiques clés (ventes du jour, produits critiques, valeur du stock)
- Historique des transactions et exports
- Module d'administration interne

4.2 Hors Périmètre

Les éléments suivants ne seront pas inclus dans la version initiale du projet :

- Application mobile native (iOS / Android) – évolution future envisagée
- Intégration avec des systèmes de comptabilité externes (Sage, etc.)
- Gestion multi-magasins ou franchise
- Module e-commerce ou livraison
- Intégration de systèmes de paiement électronique (CMI, PayZone)
- Intelligence artificielle pour la prédiction des ventes

4.3 Contraintes du Projet

Type	Contrainte	Justification
Technique	NestJS + React + PostgreSQL	Déjà initialisée dans le dépôt GitHub
UX	Interface simple et intuitive	Cible : commerçants non-techniciens
Langue	Application en français	Contexte marocain francophone
Sécurité	Protection des données commerciales	Données sensibles (ventes, marges)

5. Identification des Acteurs

La plateforme Moul Hanout est conçue pour interagir avec les types d'acteurs suivants :

Acteur	Profil	Rôle principal
Gérant	Propriétaire de l'épicerie	Accès complet : gestion des produits, stocks, alertes, rapports, utilisateurs, Gestion des comptes
Caissier	Employé chargé de la caisse	Accès au POS uniquement : enregistrer les ventes et consulter le stock
Technicien (optionel)	Gestionnaire technique de la plateforme	supervision technique, maintenance

6. Description Générale du Système

Moul Hanout est une application web (SPA) développée en React pour l'interface utilisateur, reposant sur une API RESTful construite avec NestJS. La persistance des données est assurée par PostgreSQL, accédé via l'ORM Prisma. L'ensemble de l'infrastructure est conteneurisée avec Docker pour garantir la reproductibilité des environnements de développement et de production.

Le système repose sur une architecture en trois grandes couches :

- Couche présentation (Frontend React) : interface POS, tableaux de bord, gestion des produits et des alertes
- Couche applicative (Backend NestJS) : logique métier, API REST, gestion des rôles et des permissions, déclenchement des alertes
- Couche données (PostgreSQL + Prisma) : stockage persistant des produits, transactions, stocks, alertes et utilisateurs

Le flux principal de fonctionnement est le suivant : le caissier scanne ou sélectionne des produits dans l'interface POS, valide la vente, ce qui déclenche automatiquement la mise à jour du stock. Le système vérifie en temps réel si un seuil d'alerte est atteint et notifie l'admin via le tableau de bord et par notification interne. L'Admin peut, depuis son espace dédié, consulter les rapports, gérer le catalogue et configurer les seuils d'alerte.

7. Besoins Fonctionnels

Les besoins fonctionnels décrivent l'ensemble des fonctionnalités que le système doit offrir pour répondre aux attentes des utilisateurs identifiés. Ils sont organisés par module métier.

7.1 Module d'Authentification et Gestion des Comptes

Fonctionnalités

- Inscription et création de compte pour le Gérant
- Connexion sécurisée avec email et mot de passe
- Gestion des tokens JWT (access token + refresh token)
- Déconnexion et invalidation de session
- L'Admin peut créer des comptes Caissier et les désactiver

Règles de Gestion

ID	Règle de Gestion	Type
RG01	Un utilisateur doit être authentifié pour accéder à toute fonctionnalité du système	Métier
RG02	Chaque compte doit être associé à une adresse e-mail unique	Métier
RG03	Les mots de passe doivent être hachés avec bcrypt avant stockage	Technique
RG04	Le refresh token est stocké en base et invalidé lors de la déconnexion	Technique
RG05	Seul l'Admin peut créer et désactiver des comptes Caissier	Métier

7.2 Module de Gestion du Catalogue Produits

Fonctionnalités

- Créer, modifier et archiver des produits
- Associer chaque produit à une catégorie (ex. : boissons, épicerie sèche, produits frais)
- Saisir : nom, description, code-barres, prix de vente, prix d'achat, unité, photo
- Définir un seuil d'alerte de stock minimal par produit
- Définir une date de péremption par lot (batch)
- Rechercher et filtrer les produits par nom, catégorie ou code-barres

Règles de Gestion

ID	Règle de Gestion	Type
RG06	Tout produit doit avoir un nom, un prix de vente et une catégorie	Métier
RG07	Le code-barres doit être unique s'il est renseigné	Métier
RG08	Le prix d'achat doit être inférieur ou égal au prix de vente	Métier
RG09	Un produit archivé n'est plus affiché dans le POS mais ses données sont conservées	Métier
RG10	Seul l'Admin peut créer, modifier ou archiver un produit	Métier

7.3 Module Point de Vente (POS)

Fonctionnalités

- Interface de caisse claire avec recherche rapide de produits par nom ou code-barres
- Panier de vente : ajout, suppression, modification des quantités
- Calcul automatique du total TTC et de la monnaie à rendre
- Enregistrement de la transaction avec horodatage
- Impression ou aperçu du ticket de caisse
- Gestion des paiements en espèces (en priorité) et annulation de vente

Règles de Gestion

ID	Règle de Gestion	Type
RG11	Une vente ne peut être enregistrée que si le stock est suffisant pour chaque produit	Métier
RG12	Chaque transaction est enregistrée avec la date, l'heure, l'utilisateur et le détail des articles	Technique
RG13	Le stock est décrémenté automatiquement à la validation d'une vente	Métier
RG14	Une vente validée ne peut être modifiée ; elle peut uniquement être annulée par le admin	Métier
RG15	Le Caissier a accès au POS mais ne peut pas consulter les rapports financiers	Métier

7.4 Module de Gestion des Stocks (Inventaire Entrées/Sorties)

Fonctionnalités

- Consulter le niveau de stock actuel de chaque produit
- Enregistrer une entrée de stock (approvisionnement) avec la quantité, la date et le fournisseur
- Enregistrer une sortie manuelle (perte, casse, ajustement)
- Historique complet des mouvements de stock par produit
- Vue globale du stock (tableau de bord stock avec valeur totale estimée)

Règles de Gestion

ID	Règle de Gestion	Type
RG16	Tout mouvement de stock (entrée ou sortie) est horodaté et attribué à un utilisateur	Technique
RG17	Le stock ne peut pas être négatif sauf exception validée par le Gérant	Métier
RG18	Les entrées de stock doivent obligatoirement mentionner la quantité et la date	Métier
RG19	La valeur du stock est calculée dynamiquement (quantité × prix d'achat)	Technique
RG20	Seul l'Admin peut saisir des ajustements manuels de stock	Métier

7.5 Module d'Alertes

7.5.1 Alertes de Rupture de Stock

- Chaque produit dispose d'un seuil d'alerte configurable (ex. : 5 unités)
- Lorsqu'un produit atteint ou passe sous ce seuil, une alerte est générée
- Les alertes sont visibles dans le tableau de bord du Gérant
- Un badge de notification indique le nombre d'alertes actives

7.5.2 Alertes de Date de Péremption

- Lors de l'enregistrement d'un lot, la date de péremption est saisie
- Le système génère des alertes à J-7, J-3 et J-1 avant expiration
- Les produits expirés sont marqués distinctement dans l'interface
- L'Admin reçoit une notification interne récapitulative

Règles de Gestion

ID	Règle de Gestion	Type
RG21	Une alerte de stock bas est déclenchée automatiquement à chaque vente si le stock atteint le seuil	Métier
RG22	Le seuil d'alerte est configurable par produit par l'Admin uniquement	Métier
RG23	Les alertes de péremption sont recalculées chaque jour via une tâche planifiée (cron job)	Technique
RG24	Une alerte lue reste consultable dans l'historique des alertes	Métier
RG25	Les produits expirés ne peuvent plus être vendus depuis le POS	Métier

7.6 Module Tableau de Bord et Rapports

Fonctionnalités

- Tableau de bord principal : chiffre d'affaires du jour / semaine / mois, nombre de transactions, produits les plus vendus
- Rapport des stocks critiques (rupture imminente) et des produits périmés
- Rapport des ventes par période, par catégorie ou par produit
- Export des données en CSV ou PDF
- Visualisation graphique des tendances de vente

Règles de Gestion

ID	Règle de Gestion	Type
RG26	L'accès aux rapports financiers est réservé à l'Admin	Métier
RG27	Les données des rapports sont calculées en temps réel depuis la base de données	Technique
RG28	L'export est disponible au format CSV et PDF	Technique

7.7 Module Administration

Fonctionnalités

- Gestion des comptes utilisateurs (création, activation, désactivation)
- Consultation des journaux d'activité (logs)
- Configuration des paramètres généraux de l'épicerie (nom, logo, devise)
- Supervision de la santé du système (endpoint /health)

Règles de Gestion

ID	Règle de Gestion	Type
RG29	L'administrateur peut consulter tous les journaux d'activité sans restriction	Métier
RG30	Les paramètres généraux ne peuvent être modifiés que par l'admin	Métier
RG31	Un compte désactivé ne peut plus se connecter mais ses données sont conservées	Métier

8. User Stories

8.1 User Stories – Gérant

ID	User Story
US-G01	En tant que Gérant, je veux me connecter avec mon email et mon mot de passe afin d'accéder à mon espace de gestion.
US-G02	En tant que Gérant, je veux créer un compte Caissier afin de lui donner accès au POS.
US-G03	En tant que Gérant, je veux ajouter un nouveau produit au catalogue avec son prix, sa catégorie et son seuil d'alerte afin de le rendre disponible à la vente.
US-G04	En tant que Gérant, je veux modifier les informations d'un produit existant afin de maintenir le catalogue à jour.
US-G05	En tant que Gérant, je veux enregistrer une entrée de stock afin de refléter un approvisionnement.
US-G06	En tant que Gérant, je veux configurer le seuil d'alerte de chaque produit afin d'être notifié avant la rupture de stock.
US-G07	En tant que Gérant, je veux saisir la date de péremption d'un lot lors de l'approvisionnement afin que le système génère les alertes automatiquement.
US-G08	En tant que Gérant, je veux consulter le tableau de bord afin d'avoir une vue d'ensemble sur les ventes et les alertes du jour.
US-G09	En tant que Gérant, je veux consulter le rapport des produits en rupture imminente afin de planifier mes achats.
US-G10	En tant que Gérant, je veux exporter le rapport des ventes en CSV afin de l'analyser dans Excel.
US-G11	En tant que Gérant, je veux recevoir une alerte visuelle dans le tableau de bord lorsqu'un produit est à J-3 de sa date de péremption afin de décider de son retrait.
US-G12	En tant que Gérant, je veux annuler une vente afin de corriger une erreur de caisse.

8.2 User Stories – Caissier

ID	User Story
US-C01	En tant que Caissier, je veux me connecter afin d'accéder à l'interface POS.
US-C02	En tant que Caissier, je veux rechercher un produit par nom ou code-barres afin de l'ajouter rapidement au panier.
US-C03	En tant que Caissier, je veux voir le total calculé automatiquement afin de demander le montant correct au client.
US-C04	En tant que Caissier, je veux saisir le montant remis par le client afin que la monnaie à rendre soit calculée automatiquement.
US-C05	En tant que Caissier, je veux valider la vente afin que le stock soit mis à jour et la transaction enregistrée.
US-C06	En tant que Caissier, je veux imprimer ou afficher un ticket de caisse afin de le remettre au client.
US-C07	En tant que Caissier, je veux être informé si un produit est en rupture de stock afin d'en informer le client.
US-C08	En tant que Caissier, je veux me déconnecter en fin de service afin de sécuriser l'accès à la caisse.

8.3 User Stories – Système (Alertes Automatiques)

ID	User Story
US-S01	En tant que système, je génère une alerte de stock bas après chaque vente si le stock d'un produit atteint ou passe sous son seuil.

US-S02	En tant que système, je vérifie chaque jour les dates de péremption et génère des alertes à J-7, J-3 et J-1.
US-S03	En tant que système, je bloque la vente d'un produit expiré dans le POS et notifie le Caissier.
US-S04	En tant que système, je calcule et mets à jour la valeur du stock en temps réel après chaque mouvement.

9. Besoins Non Fonctionnels

9.1 Sécurité

- Authentification par JWT (access token court + refresh token longue durée)
- Hachage des mots de passe avec bcrypt (salt rounds ≥ 10)
- Contrôle d'accès basé sur les rôles (RBAC) sur chaque endpoint API
- Protection contre les injections SQL via Prisma (requêtes paramétrées)
- Utilisation de HTTPS en production
- Variables d'environnement pour les secrets (jamais hardcodées)

9.2 Performance

- Temps de réponse des API REST ≤ 500 ms pour 95 % des requêtes en charge normale
- L'interface POS doit être opérationnelle en moins de 2 secondes au chargement
- Le calcul des alertes (cron job) doit s'exécuter en moins de 10 secondes
- Support de connexions simultanées pour un usage multi-postes dans un même magasin

9.3 Disponibilité et Fiabilité

- Disponibilité cible en production : 99,5 % (hors maintenance planifiée)
- Mécanisme de sauvegarde automatique quotidienne de la base de données
- Gestion des erreurs centralisée avec logs structurés côté backend
- Endpoint /health pour la supervision de l'état du système

9.4 Ergonomie et Accessibilité

- Interface intuitive ne nécessitant pas de formation préalable pour le Caissier
- Design responsive compatible bureau et tablette
- Temps d'apprentissage pour le Caissier estimé à moins de 30 minutes
- Affichage des alertes critique bien visible (couleurs contrastées, badges)
- Application disponible en langue française

9.5 Maintenabilité et Évolutivité

- Architecture NestJS en modules pour faciliter l'ajout de nouvelles fonctionnalités
- ORM Prisma permettant les migrations de base de données sans perte de données
- Infrastructure Docker pour faciliter le déploiement et les mises à jour
- Couverture de tests : tests unitaires et e2e sur les modules critiques (auth, stock, alertes)
- Documentation de l'API REST (Swagger) générée automatiquement

9.6 Compatibilité

- Compatible avec les navigateurs modernes : Chrome, Firefox, Edge (dernières versions)
- Compatible avec les écrans de bureau et tablettes (résolution min. 1024 px)
- Fonctionne en réseau local (LAN) sans dépendance internet obligatoire

10. Architecture Technique

10.1 Vue d'Ensemble

L'architecture de Moul Hanout suit un modèle en couches séparées communicant via des API REST. Elle est entièrement conteneurisée avec Docker pour garantir la portabilité et la facilité de déploiement.

10.2 Couche Présentation (Frontend)

Le frontend est développé avec React (TypeScript). Il s'agit d'une Single Page Application (SPA) qui consomme l'API REST du backend. Les principaux composants sont : l'interface POS, le tableau de bord, la gestion des produits, des stocks et des alertes.

- Framework : React 18 + TypeScript
- Gestion d'état : React Query (cache et synchronisation avec l'API)
- UI : Tailwind CSS + composants personnalisés
- Communication : API REST JSON via Fetch/Axios

10.3 Couche Applicative (Backend)

Le backend est développé avec NestJS (TypeScript), un framework Node.js modulaire. Il expose une API REST sécurisée et gère la logique métier, les droits d'accès et les tâches planifiées.

- Framework : NestJS (TypeScript)
- ORM : Prisma (migrations, modèles typés, requêtes sécurisées)
- Authentification : JWT (Passport.js + stratégie Bearer)
- Tests : Jest (unitaires + e2e)

10.4 Couche Données

La base de données relationnelle PostgreSQL assure la persistance des données. Prisma gère l'intégrité référentielle et les migrations.

- SGBD : PostgreSQL 15
- ORM : Prisma (schéma déclaratif, migrations versionées)
- Sauvegarde : pg_dump automatisé (tâche planifiée)

10.5 Infrastructure et DevOps

- Conteneurisation : Docker + Docker Compose (services : backend, frontend, PostgreSQL)
- CI/CD : GitHub Actions (lint, tests, build automatiques sur chaque push)
- Gestion du code : Git (monorepo avec workspaces npm)
- Variable d'environnement : fichier .env (non versionné, .env.example fourni)

11. Modélisation de la Base de Données

11.1 Entités Principales

La base de données repose sur les entités suivantes, reliées selon les cardinalités métier du système.

Entité	Principaux Attributs	Description
User	id, email, passwordHash, role (OWNER/CASHIER), isActive, createdAt	Compte utilisateur du système (Admin ou Caissier)
Session	id, userId, refreshToken, expiresAt, createdAt	Gestion des sessions actives et refresh tokens
Category	id, name, description	Catégorie de produit (ex. : boissons, épicerie sèche)
Product	id, name, barcode, sellingPrice, purchasePrice, unit, categoryId, alertThreshold, isArchived	Produit du catalogue de l'épicerie
StockBatch	id, productId, quantity, expiryDate, supplierId, receivedAt	Lot d'approvisionnement avec date de péremption et quantité
StockMovement	id, productId, type (IN/OUT/ADJUST), quantity, reason, userId, createdAt	Journal de tous les mouvements de stock
Sale	id, userId, totalAmount, paymentMethod, status, createdAt	Transaction de vente enregistrée au POS
SaleItem	id, saleId, productId, quantity, unitPrice, subtotal	Ligne de détail d'une vente (produit + quantité + prix)
Alert	id, type (LOW_STOCK/EXPIRY), productId, batchId, message, isRead, createdAt	Alerte générée par le système (stock bas ou péremption)

11.2 Relations Clés

- Un Produit appartient à une Catégorie (N-1)
- Un Produit peut avoir plusieurs Lots d'approvisionnement (1-N)
- Un Produit peut avoir plusieurs Mouvements de stock (1-N)
- Une Vente est composée de plusieurs Lignes de vente (1-N)
- Une Alerte est associée à un Produit et optionnellement à un Lot (N-1)
- Une Session est associée à un Utilisateur (N-1)

12. Planification du Projet

12.1 Phases du Projet

Phase	Activités principales	Livrable
Phase 1 – Analyse & Cadrage	Étude du problème, identification des acteurs, rédaction du CDC	Cahier des Charges
Phase 2 – Conception	Modélisation UML, schéma de base de données, maquettes UI	Schéma DB, Maquettes
Phase 3 – Fondation Backend	Auth, users, sessions, Prisma schema, seed data, CI/CD	API Auth + CI
Phase 4 – Catalogue & Stock	CRUD produits, catégories, mouvements de stock, fournisseurs	API Produits + Stock
Phase 5 – Module POS	Interface caisse, panier, validation vente, ticket, mise à jour stock	POS fonctionnel
Phase 6 – Alertes	Alertes stock bas, alertes péremption, cron job, notifications UI	Module Alertes
Phase 7 – Dashboard & Rapports	Tableau de bord, graphiques, exports CSV/PDF	Rapports + Export
Phase 8 – Tests & Validation	Tests unitaires, tests e2e, recette fonctionnelle, corrections	Application testée
Phase 9 – Documentation	Documentation technique, README, rapport final PFA	Rapport PFA

12.2 Estimation Temporelle

Phase	Durée estimée	Remarques
Phase 1 – Analyse & Cadrage	0.5 semaine	
Phase 2 – Conception	0.5 semaine	Revue des maquettes
Phase 3 – Fondation Backend	1 semaine	Base déjà initialisée sur GitHub
Phase 4 – Catalogue & Stock	1,5 semaine	
Phase 5 – Module POS	1,5 semaines	Phase la plus complexe
Phase 6 – Alertes	0.5 semaine	
Phase 7 – Dashboard & Rapports	1 semaine	
Phase 8 – Tests & Validation	1 semaine	
Phase 9 – Documentation	0,5 semaine	
TOTAL	**~ 8 semaines**	

13. Gestion des Risques

Les principaux risques identifiés pour le projet Moul Hanout, leur impact potentiel et les mesures de mitigation associées sont présentés dans le tableau suivant.

Risque identifié	Impact potentiel	Mesures de mitigation
Complexité du module POS (gestion concurrente du stock)	Incohérence des stocks lors de ventes simultanées	Transactions Prisma atomiques, verrouillage optimiste des lignes de stock
Faux positifs / négatifs dans les alertes péremption	Alertes manquantes ou trop nombreuses, perte de confiance du commerçant	Tests unitaires exhaustifs du cron job, logs détaillés de chaque exécution
Retard dans le développement	Non-respect du planning PFA	Planning par sprints hebdomadaires, points d'avancement réguliers
Sécurité des données commerciales	Accès non autorisé aux ventes et marges	RBAC strict, JWT sécurisés, HTTPS obligatoire en production
Performances insuffisantes du POS	Expérience dégradée, lenteur en caisse	Optimisation des requêtes Prisma, index DB sur produits et stock
Problèmes d'intégration Frontend/Backend	Blocages lors de l'assemblage des modules	Définir les contrats d'API (Swagger) dès la Phase 2, tests e2e automatisés
Perte ou corruption des données	Perte des historiques de ventes et de stock	Sauvegardes automatiques quotidiennes, tests de restauration
Manque d'adoption par les commerçants	Application non utilisée en pratique	Tests utilisateurs avec de vrais gérants d'épicerie, interface ultra-simplifiée

14. Conclusion

Le présent Cahier des Charges définit de manière exhaustive les contours du projet Moul Hanout, une solution de digitalisation pensée pour les épiceries et commerces de proximité marocains. Il couvre l'ensemble des dimensions du projet : contexte et problématique, objectifs, périmètre fonctionnel, besoins détaillés, architecture technique, modélisation des données, planification et gestion des risques.

Le projet s'appuie sur une stack technique moderne et éprouvée (NestJS, React, PostgreSQL, Prisma, Docker), ce qui constitue un avantage significatif pour le respect du calendrier académique. Les modules prioritaires – POS, gestion des stocks, alertes – répondent directement aux douleurs les plus critiques des commerçants cibles.

Au-delà du cadre académique, Moul Hanout ambitionne de devenir une véritable solution viable pour les petits commerçants marocains. Des évolutions futures pourront inclure une application mobile, le support de la carte de fidélité, ou encore des fonctionnalités de prédiction de stock basées sur l'intelligence artificielle.

Ce document constitue la référence partagée entre l'équipe de développement et l'encadrant académique, et sera mis à jour en cas d'évolution du périmètre du projet.

15. Conception – Diagrammes UML

Les diagrammes ci-dessous modélisent les flux et la structure du Moul Hanout. Ils couvrent uniquement le périmètre défini dans ce cahier des charges : une seule boutique, deux rôles (Admin / Caissier), gestion de produits et catégories, gestion du stock, vente en espèces, et tableau de bord opérationnel.

15.1 Diagramme de Cas d'Utilisation (Use Case)

Ce diagramme représente les interactions entre les acteurs (Admin, Caissier) et le système pour l'ensemble des cas d'utilisation du .

Acteurs :

- Admin : se connecter, gérer les catégories, gérer les produits, gérer le stock (entrée / sortie), consulter le tableau de bord.
- Caissier : se connecter, consulter le stock, créer une vente, ajouter des articles au panier, enregistrer un paiement en espèces, générer un reçu.

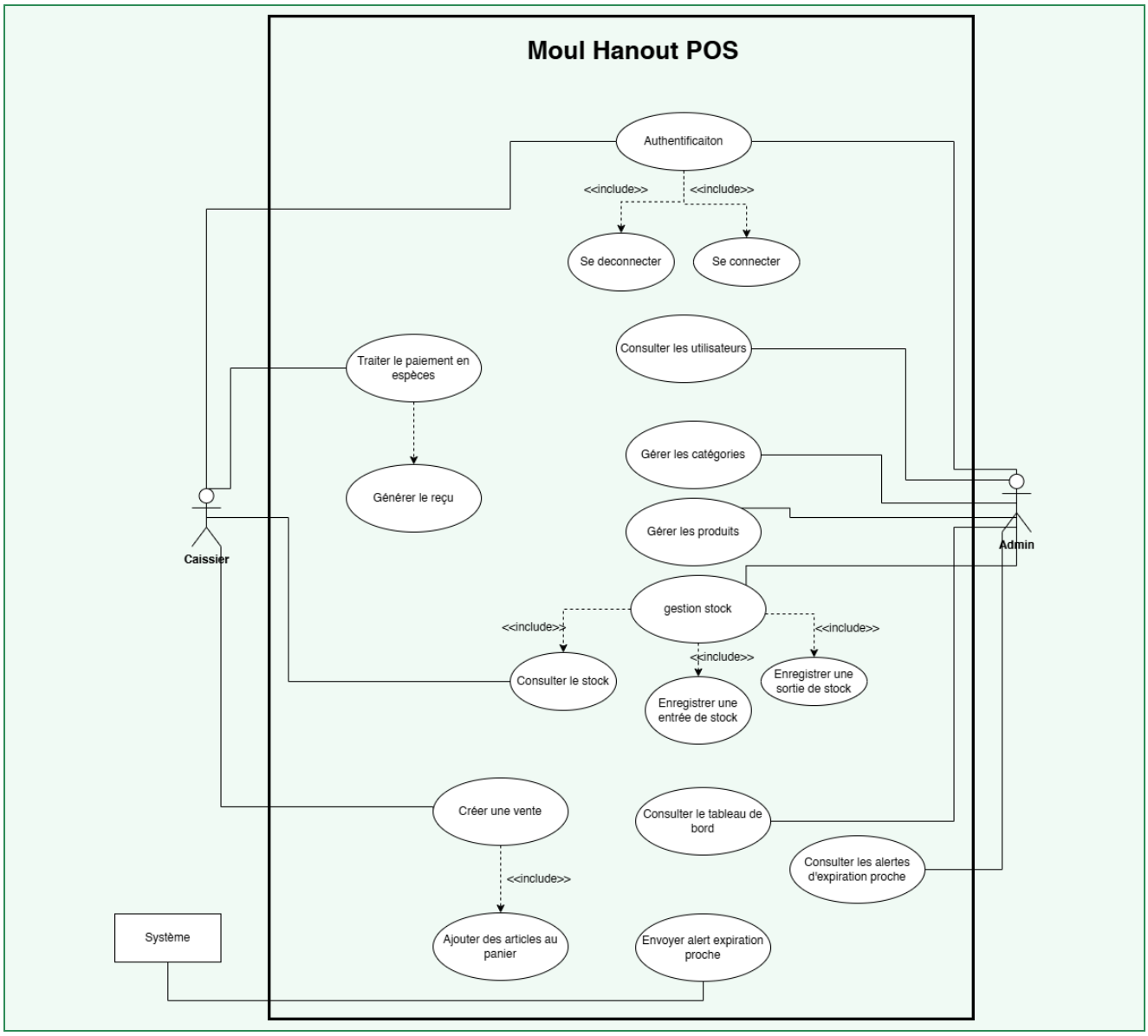

Figure 1: Diagramme de cas d'utilisation

15.2 Diagramme de Classes

Ce diagramme représente le modèle de données du . Les entités principales sont : **User, Role, Category, Product, StockMovement, Sale, SaleItem, Payment, AuditLog**. Le stock reste sur **Product.currentStock** pour le , l'historique étant tracé dans StockMovement.

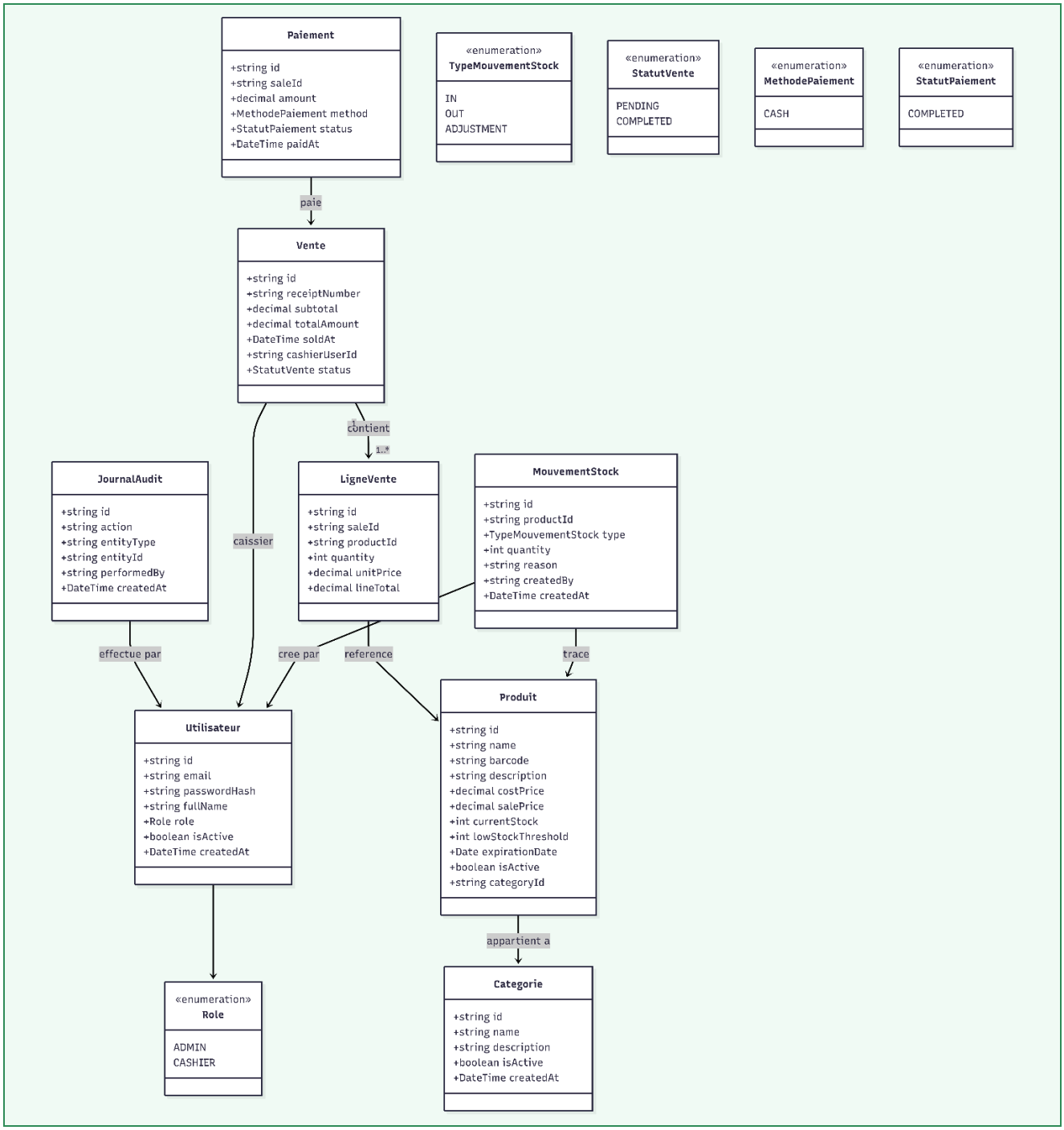

Figure 2: Diagramme des classes

15.3 Diagramme de Séquence – Connexion (Login)

Ce diagramme décrit le flux de connexion JWT du . Participants : **Utilisateur (Admin ou Caissier), Frontend, AuthController, AuthService, UsersService, Base de données**. Le flux couvre : soumission du formulaire → validation des identifiants → vérification du mot de passe haché → génération du JWT → redirection selon le rôle.

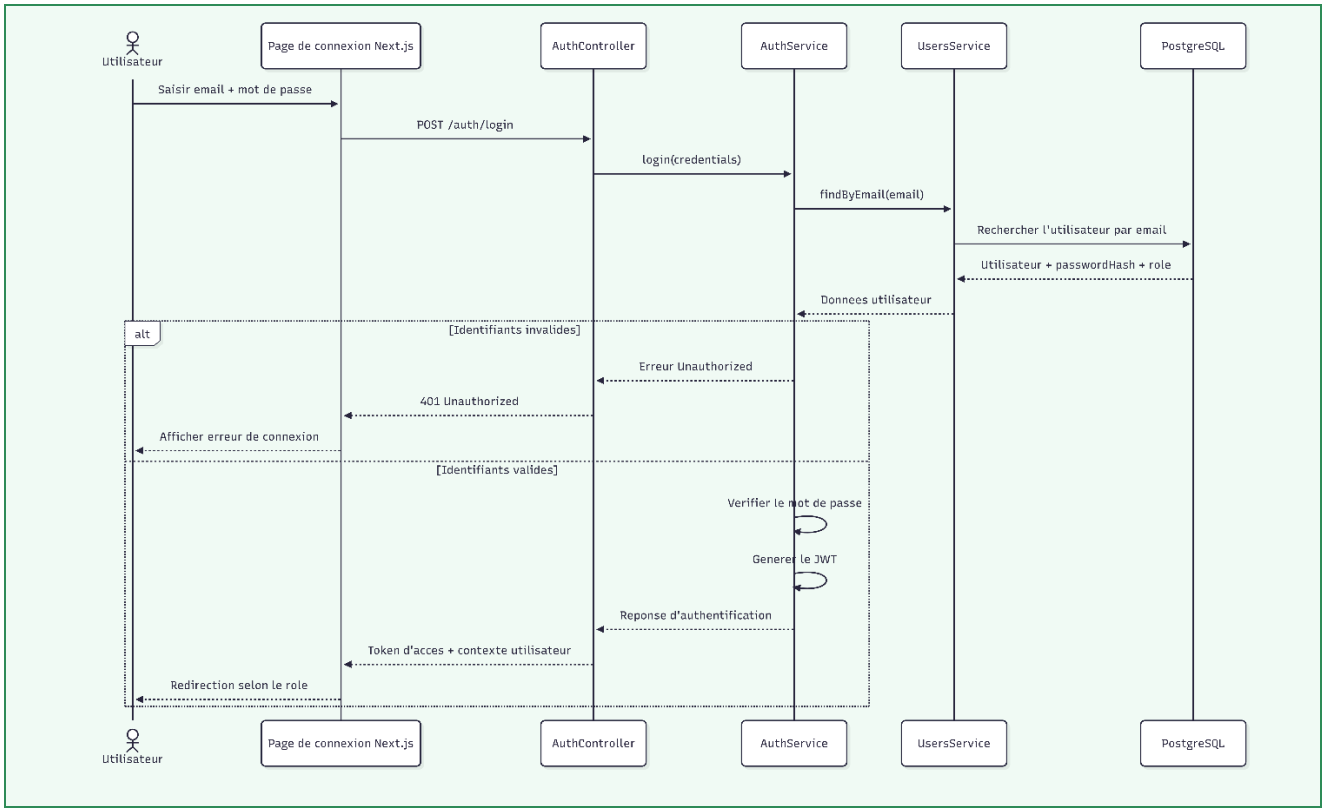

Figure 3: Diagramme de Séquence – Connexion (Login)

15.4 Diagramme de Séquence – Vente, Paiement et Reçu

Ce diagramme couvre le flux complet de caisse du , de la création de la vente à la génération du reçu. Participants : **Caissier, Frontend, SalesController, SalesService, PaymentsService, InventoryService, Base de données**. Le flux couvre : création de la vente → ajout des articles → validation du stock → confirmation du paiement en espèces → décrémentation automatique du stock (dans la même transaction) → génération du reçu.

Étapes principales du flux :

1. Le Caissier démarre une nouvelle vente (POST /sales).
2. Le Caissier ajoute des produits et leurs quantités au panier.
3. Le backend valide la disponibilité du stock pour chaque article.
4. Le backend calcule les totaux (lignes + total général).
5. Le Caissier confirme le paiement en espèces (méthode : CASH uniquement).
6. Le backend enregistre le paiement, finalise la vente et décrémente le stock dans la même transaction Prisma.
7. Le backend enregistre les mouvements de stock (type : OUT) dans StockMovement.
8. Le système retourne les données du reçu (numéro, date, caissier, articles, total, mode de paiement).

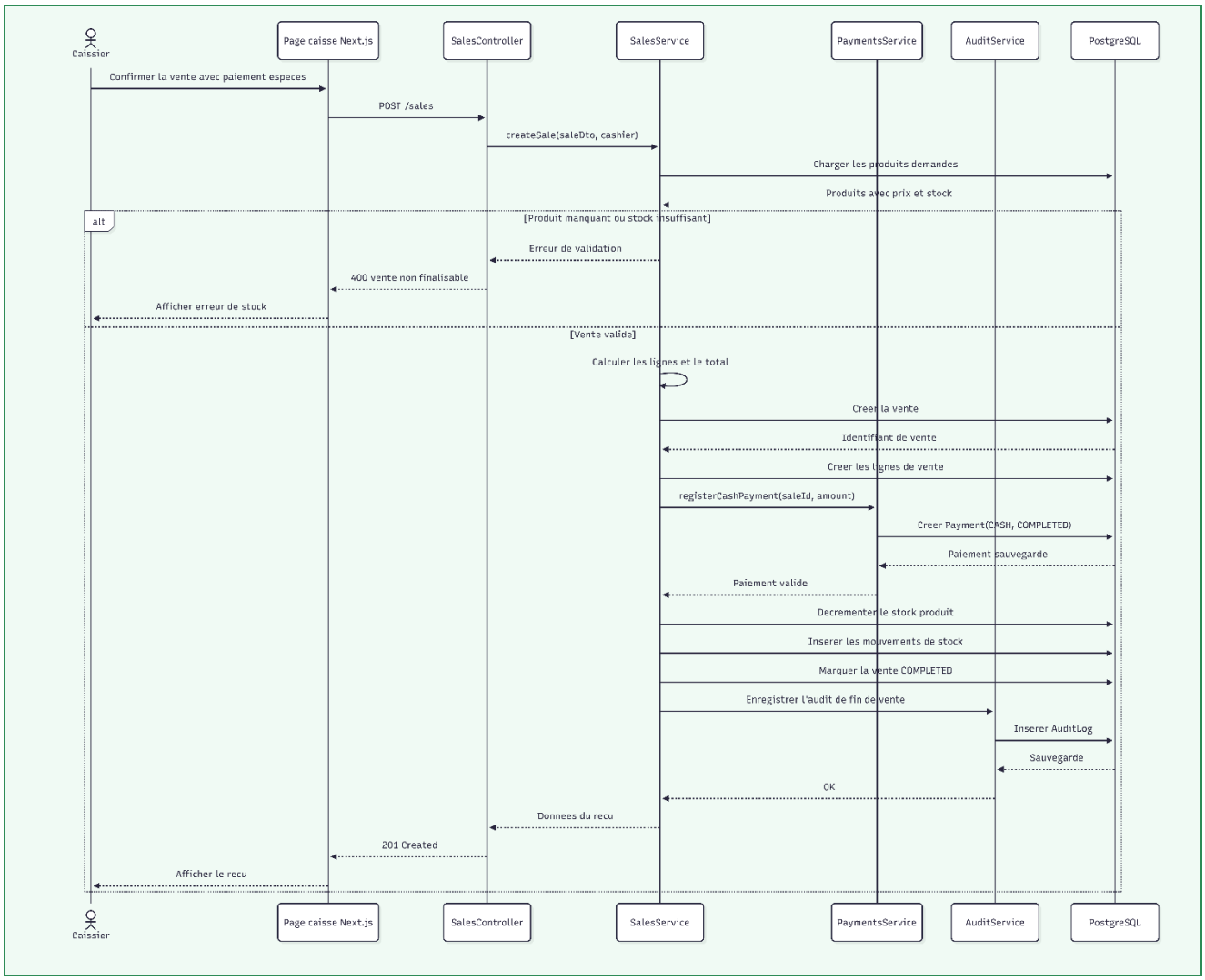

Figure 4: Diagramme de Séquence – Vente, Paiement et Reçu